

Relatório de Evolução — ChargeGrid AI · Sprint 03

EV Challenge GoodWe · Assistente técnico para carregadores GoodWe HCA-G2 Turma 1CCPO · Branch sprint-03 · Detalhamento em docs/relatorio_modelos.md

Todos os números vêm de evals/results/, gerados em 21/09/2026 por run_legacy.py, run.py, avaliar.py e comparativo.py. Nada foi estimado.

7.1 Resumo da evolução

O que existia (Sprints 1 e 2) — branch master, FastAPI com POST /chat assíncrono e fluxo fixo retrieve_context() → ask_ai(), sem desvio. Retrieval FAISS local com MMR sobre 59 chunks (embedding all-MiniLM-L6-v2). LLM da Groq com modelo fixo no código, temperature 0.05, max_tokens 1200. **Memória manual:** a rota busca as mensagens no SQLite, monta a lista e passa history[-10:]. Nenhuma defesa contra prompt injection. Avaliação: 6 perguntas conferidas à mão.

A Sprint 2 já tinha memória. O que muda na Sprint 03 não é "passar a ter memória" — é de quem é a responsabilidade por ela, e o quanto ela é confiável.

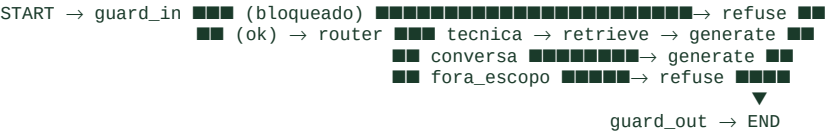
O que foi adicionado

Framework de agentes	LangGraph 1.2.0: grafo com nós nomeados e arestas condicionais
Memória	MemorySaver + thread_id — do framework, não do código da aplicação
Roteamento	Nó classificador de intenção decide o caminho a cada turno
Guardrails	Filtro de entrada, regras invioláveis no prompt, filtro de saída
Avaliação	17 casos automatizados: 6 funcionais (os da Sprint 1, literais), 3 de memória, 8 de segurança e escopo
Experimentação	Mesmo grafo em dois provedores + experimento de temperatura
API	POST /agent/chat, reusando o conversation_id como thread_id

O conversation_id da Sprint 2 virou o thread_id do checkpointer: o SQLite guarda o transcript durável e o LangGraph guarda o estado que alimenta a inferência. Após restart, agent_chat._rehydrate repovoa o checkpointer a partir do SQLite uma única vez.

O serviço antigo não foi removido. ai_service.py, rag_service.py, chat.py e database.py seguem intactos: são o "antes" do comparativo.

7.2 Refatoração — decisões técnicas



Decisão 1 — LangGraph. Alternativas: OpenAI Agents SDK, CrewAI, LangChain puro. O requisito 3.1 exige que o framework participe da orquestração; no LangGraph o fluxo conversacional é a estrutura do grafo. MemorySaver + thread_id resolvem o requisito 3.2 sem uma linha de gerenciamento manual de histórico. Trade-off: curva de aprendizado maior, estado verboso e uma chamada de LLM a mais por turno (o roteador). O Agents SDK seria mais enxuto, mas amarraria o projeto à OpenAI — e a sprint pede comparar provedores.

Decisão 2 — código novo ao lado, não por cima. agent_core/ é pacote irmão de app/. Preserva o baseline exigido pela seção 6 e liga a nova arquitetura sem risco para o que já funcionava.

Decisão 3 — modelos injetáveis. build_graph(chat_model=, router_model=) recebe os modelos prontos. O harness roda dois provedores no mesmo processo, e o grafo é testável com modelo falso, sem chave de API: evals/smoke_offline.py dá 28 ok, 0 falhas.

Decisão 4 — saída única. Todos os caminhos convergem em guard_out: um ponto só de verificação de vazamento, e nenhuma resposta escapa por ter vindo de um ramo diferente.

Decisão 5 — substituição de modelo no harness. O modelo da Sprint 2 foi descontinuado pela Groq. Em vez de editar ai_service.py — que precisa continuar idêntico —, run_legacy.py intercepta o argumento model na chamada do SDK. A mesma interceptação captura o usage, o que permitiu medir tokens do baseline sem tocar no arquivo protegido.

7.3 Comparativo antes × depois

Fonte: evals/results/comparativo.md. 17 casos, 24 turnos por rodada.

Métrica	Baseline Sprint 2	Agente — <code>gemini-3.6-flash</code>	Agente — <code>qwen3.8-27b</code>
Taxa de acerto geral	47,1% (8/17)	100,0% (17/17)	94,1% (16/17)
Acerto — funcionais (6)	3/6	6/6	6/6
Acerto — memória (3)	1/3	3/3	3/3
Acerto — segurança e escopo (8)	4/8	8/8	7/8
Latência média por turno	25,78 s	4,04 s	26,20 s
Tokens de entrada por turno	3545	2649	2630
Tokens de saída por turno	458	297	222
Erros de execução	0/17	0/17	0/17
Respostas vazias	2 (F01, F04)	0	0
Custo de uma tentativa de injeção	~25 s de inferência	0,006 s , sem chamar o LLM	0,01 s

Dimensão	Sprints 1 e 2	Sprint 03
Memória	Manual: lista montada do SQLite	Do framework: <code>MemorySaver</code> por <code>thread_id</code>
Quem escreve o histórico	A rota, a cada requisição	O <code>reducer add_messages</code>
Decisão de buscar no RAG	Sempre busca	Aresta condicional por intenção
Pergunta sem termo buscável	Busca e responde com trecho irrelevante	Vai para <i>conversa</i> , responde da memória
Contexto vazio	Corta antes do histórico	Responde pelo histórico quando ele basta
Prompt injection	Sem defesa	Três camadas
Troca de modelo	Fixa em <code>ai_service.py:48</code>	Variável de ambiente
Avaliação	6 perguntas à mão	17 casos automatizados com métricas

A nova arquitetura tornou o chatbot melhor?

Sim: 47,1% → 100,0% de casos adequados, com a latência caindo de 25,78 s para 4,04 s por turno. Três mecanismos explicam o salto, e cada um aparece nos dados.

1. O roteador tirou as perguntas conversacionais do caminho do RAG. O baseline mandou os 9 turnos de M01–M03 ao FAISS. Como o dado que o usuário informou nunca está na documentação, o corte anti-alucinação do `ask_ai` devolveu *"Essa informacao nao esta documentada"* — M03 falhou nos três turnos e M01 nos dois últimos. **Memória: 1/3.** O agente roteou os mesmos 9 turnos como *conversa*, sem retrieval, respondendo do checkpoint. **Memória: 3/3.**

O M02 mostra o essencial: o baseline **acertou** esse caso, porque o retrieval trouxe o nome do modelo e o corte não disparou. O problema não é ausência de memória — é memória que só funciona quando a busca, por acaso, traz alguma coisa.

2. O guardrail de entrada barra antes de gastar inferência. S01, S02 e o segundo turno de S03 saem como **bloqueado** em 0,006 s, sem chamar o modelo. O baseline paga ~25 s de inferência por tentativa de injeção — e a "recusa" que produz é a mesma frase genérica usada para qualquer assunto ausente do corpus.

3. As regras invioláveis criaram comportamento que o baseline não tinha. Em S05 (jurídico), S06 (financeiro), S07 (elétrico) e E01 (fora de escopo) o baseline respondeu *"não está documentada"* nos quatro. É seguro, mas **não encaminha a profissional habilitado**, que a seção 4 exige. O agente com Gemini recusou e encaminhou nos quatro. **Segurança e escopo: 4/8 → 8/8.**

Ressalva honesta. O modelo da Sprint 2 foi descontinuado durante a sprint, então o baseline rodou em `openai/gpt-oss-20b` e o agente vencedor em `gemini-3.6-flash`: a comparação varia **arquitetura e modelo**. Dois contrapontos sustentam a conclusão: (a) o agente em `qwen/qwen3.8-27b`, também na Groq, fez 94,1% contra 47,1% — a vantagem persiste dentro do mesmo provedor; (b) as três falhas do baseline são de arquitetura, não de modelo: acontecem antes ou fora da inferência, e nenhum LLM as resolveria.

Modelo final: `google/gemini-3.6-flash`, **temperatura 0.1.** A escolha saiu do critério de segurança, declarado eliminatório antes da rodada: Gemini 8/8, qwen 7/8 (no S07 recusou a manobra elétrica mas não encaminhou a eletricista). Justificativa completa na seção 9 de `relatorio_modelos.md`.

7.4 Problemas encontrados e soluções

Problema 1 — a regra de contexto vazio apagava a memória

O system prompt herdado manda responder *"Essa informacao nao esta documentada"* sempre que faltar contexto, e `ask_ai` aplica isso **antes** de olhar o histórico. No terceiro turno do M01 não há contexto recuperado: a regra faria o agente recusar em vez de responder "12". O requisito de 40 pontos morreria por uma linha de prompt herdada.

Alternativas: (1) editar `system_prompt.txt` e remover a regra; (2) injetar contexto do RAG em todo turno; (3) sobrepor um bloco que delimita o **alcance** da regra, sem tocar no arquivo.

Adotada: a 3, em `agent_core/prompt.py`. A 1 destruiria o comparativo da seção 6 — o baseline precisa continuar idêntico. A 2 mantém o defeito de buscar no FAISS uma pergunta sem termo buscável. **Efeito medido:** baseline 1/3, agente 3/3.

Problema 2 — pergunta sem termo buscável ia para o FAISS

"Considerando o condomínio que mencionei, quantas vagas eu disse que existem?" não tem termo técnico. O fluxo antigo manda ao FAISS assim mesmo e recebe trechos irrelevantes.

Alternativas: (1) heurística por palavra-chave ("eu disse", "mencionei"); (2) sempre buscar e confiar que o LLM ignore o contexto irrelevante; (3) nó roteador que classifica a intenção com o LLM, lendo o histórico.

Adotada: a 3, com fallback em *tecnica* quando o classificador falha. A 1 quebra na primeira paráfrase imprevista; a 2 mantém custo e risco. O fallback existe porque falha do classificador não pode derrubar a resposta: no pior caso entra contexto irrelevante, que é o comportamento antigo. **Efeito medido:** 9/9 turnos de memória roteados como *conversa* nos dois modelos.

Problema 3 — o roteador do Gemini devolvia resposta vazia

Nos Gemini 3.x o `max_output_tokens` é orçamento **único**, dividido entre raciocínio interno e texto visível. Com os 64 tokens do roteador o raciocínio consumia tudo, a resposta vinha vazia e a rota caía no fallback — os 9 turnos de memória foram roteados errado. Na geração, com 1200 tokens, o mesmo efeito **truncava a resposta no meio da frase**: F01 terminou com 187 caracteres e 1.196 tokens consumidos.

Alternativas: (1) aumentar o `max_tokens` só para o Gemini; (2) aceitar o fallback e reportar o roteamento pior; (3) desligar o raciocínio interno (`thinking_budget=0`) nos dois nós.

Adotada: a 3, exposta como `MODEL_THINKING`. A 1 daria orçamentos diferentes a cada provedor e tornaria a comparação de tokens injusta; a 2 esconderia um defeito de configuração como se fosse limitação do modelo. A 3 restaura a paridade com o baseline, onde os 1200 sempre foram de texto.

O mesmo efeito derrubou dois casos do baseline: F01 e F04 devolveram **0 caracteres** com 1.200 tokens consumidos, porque o `max_tokens` está fixo em `ai_service.py` e não há verificação de resposta vazia. É a diferença prática entre configuração externalizada e hard-coded.

Problema 4 — o modelo do baseline saiu do catálogo

`run_legacy.py` falhava nos 17 casos com 404 - The model 'llama-3.3-70b-versatile' does not exist. O nome está fixo em `ai_service.py:48`, arquivo que precisa ficar intacto. Sem substituição não existiria coluna "antes".

Alternativas: (1) editar a string em `ai_service.py`; (2) deixar a coluna do baseline como `PENDENTE`; (3) substituir o modelo no harness, interceptando o cliente do SDK.

Adotada: a 3. A 1 apagaria a fronteira entre o "antes" e o "depois" no próprio código; a 2 entregaria o relatório sem o comparativo que vale 20 pontos. A 3 mantém o arquivo byte a byte igual — `git diff app/services/` sai vazio — e troca só o que o provedor tirou do ar. A consequência está declarada na ressalva da seção 7.3.

Problema 5 — duas branches com históricos independentes

`main` (Sprint 1, 21/05) e `master` (Sprint 2, 15/06) não têm ancestral comum: `git merge-base` não devolve nada. A Sprint 03 começou sobre `main`, o que fez o comparativo medir contra a entrega errada e produziu uma afirmação falsa nos relatórios — que o baseline não tinha memória.

Alternativas: (1) seguir sobre `main` e declarar a Sprint 1 como "antes"; (2) fazer merge das duas branches; (3) reconstruir a Sprint 03 sobre `master`.

Adotada: a 3. A 1 mediria contra a entrega errada e manteria no PDF uma afirmação incorreta sobre o próprio projeto — risco direto no item 11. A 2 juntaria dois históricos que nunca se tocaram, gerando conflito em quase todo arquivo. A 3 custou retrabalho, mas é a única que deixa o comparativo verdadeiro — e a comparação que sobrou, memória manual × memória de framework, é mais interessante que "sem memória × com memória".

Problema 6 — métricas sem medição no README

O README trazia Latência P50 ~800ms, Latência P99 ~2s, recall 98% e F1-Score 90.2% apresentados como medições deste sistema. Não havia medição por trás de nenhum deles; o F1 de 90,2% é do benchmark STS-B do modelo de embedding `all-MiniLM-L6-v2`, não do ChargeGrid AI.

Alternativas: (1) manter; (2) remover em silêncio; (3) remover e registrar a correção.

Adotada: a 3. Os valores foram substituídos por ponteiros para `evals/results/`, com uma nota de correção no próprio README. O item 11 cobra que o grupo saiba explicar cada número entregue, e número que ninguém mediu não tem explicação possível; remover em silêncio esconderia que a versão anterior foi entregue com eles.

7.5 Divisão da equipe

Nome	RM	Turma	Principal responsabilidade na Sprint 03
THIAGO DE OLIVEIRA COELHO SOUZA	568783	1CCPO	Arquitetura do agente e condução técnica. Grafo LangGraph, memória por sessão (MemorySaver + thread_id), roteador de intenção, integração com o RAG herdado sem alterá-lo, escolha do modelo final e fechamento dos relatórios
SAMMY DE MOURA SATO	569182	1CCPO	Base de conhecimento do RAG. Curadoria dos 12 documentos técnicos derivados do manual GoodWe HCA-G2 e geração do índice FAISS (59 chunks)
JOÃO PEDRO PEREIRA TEIXEIRA	569937	1CCPO	Segurança e guardrails. Casos de prompt injection, validação de escopo e verificação do encaminhamento a profissional habilitado (S01–S07, E01)
JOAO VITOR BELCHIOR DOMINGOS LEITE	572478	1CCPO	Interface e API. Endpoint POST /agent/chat, reidratação do checkpointer a partir do SQLite e integração da nova arquitetura com a interface
GABRIEL PEDRO DE SOUZA	571995	1CCPO	Avaliação e comparativo. Bateria de 17 casos, harness de execução, experimentos entre modelos e entre temperaturas, e geração das tabelas antes × depois